

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Assignment Documentation

Closest Pair of Points and Convex Hull:

A Comparative Study of Brute Force, Divide-and-Conquer, and Hybrid Algorithmic Strategies

Course Code & Name: CSA0609 – Design and Analysis of Algorithms

Student Name	Vanjinathan S
Register Number	192425231
Slot	C
Faculty In-Charge	Dr. SOLAINAYAGI P
Date of Submission	03-09-2026

1. Assignment Information

1.1 Assignment Problem / Challenge

Given the set of 10 two-dimensional points $P = \{(2,3), (5,8), (9,4), (12,10), (7,2), (3,11), (15,6), (10,14), (6,12), (1,7)\}$, the Brute Force technique is applied to solve two classical computational-geometry problems: the Closest Pair of Points problem and the Convex Hull problem.

For the Closest Pair problem, the pair of points with the minimum Euclidean distance is determined by examining every possible pair of points and computing the distance using the Euclidean distance formula. The number of pairwise comparisons required, and the time and space complexity of the approach, are analysed as the number of points n increases.

For the Convex Hull problem, the Brute Force technique is used to determine the convex hull of the same point set by examining every possible candidate edge and checking the orientation of the remaining points relative to that edge. All points lying on the hull are identified, the major intermediate calculations are shown, and a graphical representation of the input points and the resulting hull is provided. A Brute Force algorithm/pseudocode is developed for the Convex Hull, and its time and space complexity is analysed, including the number of candidate edges examined.

1.2 Requirements and Constraints

A publicly available real-world coordinate dataset is used with enough points to test different input sizes, ranging from 10^3 to 10^6 points. The Brute Force, Divide-and-Conquer, and Hybrid algorithms are executed on identical inputs and in the same computational environment. Execution time and the number of operations are recorded for each input size; the crossover threshold used for the Hybrid algorithm is clearly stated, and performance trends are analysed as n increases. The submission includes the dataset source, preprocessing steps, experimental setup, algorithms/pseudocode, results, complexity analysis, graphs, assumptions, limitations, and a comparison of the three approaches.

2. Student Work

2.1 Problem Understanding and Formulation

2.1.1 Problem Analysis

Both problems belong to the field of Computational Geometry and operate on a finite set of points in the Euclidean plane.

- Closest Pair of Points: Given n points, find the two points whose straight-line (Euclidean) distance is the smallest among all $C(n,2)$ possible pairs. This is an optimisation problem over a search space that grows quadratically with n .

- **Convex Hull:** Given n points, find the smallest convex polygon that encloses every point in the set. Geometrically, a point q is on the hull boundary if and only if there exists a line through q such that all other points lie strictly on one side of it — this orientation test is the geometric primitive used by the Brute Force hull algorithm.

Both problems admit a naive Brute Force solution derived directly from their definition (exhaustive pairwise / edge-wise checking), and a more efficient Divide-and-Conquer solution that exploits geometric locality after sorting. A Hybrid strategy that switches between the two, based on a size threshold, is also required by the assignment and is implemented and benchmarked below.

2.1.2 Given Dataset

The ten input points supplied in the problem statement are:

Point	X	Y
P1	2	3
P2	5	8
P3	9	4
P4	12	10
P5	7	2
P6	3	11
P7	15	6
P8	10	14
P9	6	12
P10	1	7

2.1.3 Brute Force Algorithm / Pseudocode — Closest Pair of Points

```

ALGORITHM ClosestPair_BruteForce(P[1..n])
  minDist ← INFINITY
  closestPair ← NULL
  FOR i ← 1 TO n-1
    FOR j ← i+1 TO n
      d ← SQRT( (P[i].x - P[j].x)^2 + (P[i].y - P[j].y)^2 )
      IF d < minDist THEN
        minDist ← d
        closestPair ← (P[i], P[j])
      END IF
    END FOR
  END FOR
  RETURN closestPair, minDist
END ALGORITHM

```

The algorithm compares every unordered pair of points exactly once ($i < j$), computes the Euclidean distance, and retains the minimum found so far. It performs $C(n,2) = n(n-1)/2$ distance computations.

2.1.4 Brute Force Algorithm / Pseudocode — Convex Hull

```
ALGORITHM ConvexHull_BruteForce(P[1..n])
  hullEdges <- EMPTY SET
  FOR i <- 1 TO n
    FOR j <- 1 TO n, j != i
      pos <- 0 ; neg <- 0
      FOR k <- 1 TO n, k != i AND k != j
        o <- ORIENTATION(P[i], P[j], P[k]) // cross product sign
        IF o > 0 THEN pos <- pos + 1
        ELSE IF o < 0 THEN neg <- neg + 1
      END FOR
      // If every other point lies on a single side of line (P[i],P[j])
      // then (P[i],P[j]) is an edge of the convex hull
      IF pos == 0 OR neg == 0 THEN
        hullEdges <- hullEdges U {(P[i], P[j])}
      END IF
    END FOR
  END FOR
  hullPoints <- distinct endpoints of hullEdges, sorted angularly about centroid
  RETURN hullPoints
END ALGORITHM

FUNCTION ORIENTATION(p, q, r)
  val <- (q.y - p.y) * (r.x - q.x) - (q.x - p.x) * (r.y - q.y)
  IF val > 0 RETURN +1 // counter-clockwise turn
  IF val < 0 RETURN -1 // clockwise turn
  RETURN 0 // collinear
END FUNCTION
```

For every ordered pair of points (i, j) — $n(n-1)$ candidate directed edges in total — the algorithm checks the orientation of every other point k relative to the line through P[i] and P[j]. If all remaining points lie strictly on one side ($pos = 0$ or $neg = 0$), the segment (P[i], P[j]) is an edge of the convex hull. This runs in $O(n^3)$ because the edge loop is $O(n^2)$ and the inner orientation check is $O(n)$.

2.1.5 Step-by-Step Calculations — Closest Pair of Points

The Euclidean distance formula $d(P,Q) = \sqrt{[(x_2-x_1)^2 + (y_2-y_1)^2]}$ is applied to every one of the $C(10,2) = 45$ unique pairs of points. The full set of pairwise distances, sorted in ascending order, is tabulated below (all 45 comparisons, exactly as the Brute Force algorithm evaluates them).

Point A	Point B	Euclidean Distance
P3(9,4)	P5(7,2)	2.828
P6(3,11)	P9(6,12)	3.162
P2(5,8)	P6(3,11)	3.606
P1(2,3)	P10(1,7)	4.123
P2(5,8)	P9(6,12)	4.123
P2(5,8)	P10(1,7)	4.123
P4(12,10)	P8(10,14)	4.472

P6(3,11)	P10(1,7)	4.472
P8(10,14)	P9(6,12)	4.472
P4(12,10)	P7(15,6)	5.000
P1(2,3)	P5(7,2)	5.099
P2(5,8)	P3(9,4)	5.657
P1(2,3)	P2(5,8)	5.831
P2(5,8)	P5(7,2)	6.325
P3(9,4)	P7(15,6)	6.325
P4(12,10)	P9(6,12)	6.325
P3(9,4)	P4(12,10)	6.708
P1(2,3)	P3(9,4)	7.071
P9(6,12)	P10(1,7)	7.071
P2(5,8)	P4(12,10)	7.280
P6(3,11)	P8(10,14)	7.616
P2(5,8)	P8(10,14)	7.810
P5(7,2)	P10(1,7)	7.810
P1(2,3)	P6(3,11)	8.062
P3(9,4)	P9(6,12)	8.544
P3(9,4)	P10(1,7)	8.544
P5(7,2)	P7(15,6)	8.944
P4(12,10)	P6(3,11)	9.055
P3(9,4)	P6(3,11)	9.220
P4(12,10)	P5(7,2)	9.434
P7(15,6)	P8(10,14)	9.434
P1(2,3)	P9(6,12)	9.849
P5(7,2)	P6(3,11)	9.849
P3(9,4)	P8(10,14)	10.050
P5(7,2)	P9(6,12)	10.050
P2(5,8)	P7(15,6)	10.198
P7(15,6)	P9(6,12)	10.817
P4(12,10)	P10(1,7)	11.402
P8(10,14)	P10(1,7)	11.402
P1(2,3)	P4(12,10)	12.207
P5(7,2)	P8(10,14)	12.369
P6(3,11)	P7(15,6)	13.000
P1(2,3)	P7(15,6)	13.342

P1(2,3)	P8(10,14)	13.601
P7(15,6)	P10(1,7)	14.036

Sample calculation for the minimum-distance pair, P3(9,4) and P5(7,2):

$$d(P3,P5) = \text{SQRT}[(9-7)^2 + (4-2)^2] = \text{SQRT}[2^2 + 2^2] = \text{SQRT}[4 + 4] = \text{SQRT}[8] = 2.828$$

Closest-Pair Result: Minimum distance = 2.828 units, achieved by the pair P3(9,4) and P5(7,2). 45 comparisons were performed in total, matching the theoretical count $C(10,2)=45$.

2.1.6 Step-by-Step Calculations — Convex Hull

For every ordered pair $(P[i], P[j])$ among the 10 points, the orientation of every remaining point relative to the line through $P[i]$ and $P[j]$ was evaluated using the cross -product test. A total of $n(n-1) = 10 \times 9 = 90$ candidate directed edges were examined (45 undirected segments, each tested in both directions). An edge is part of the convex hull boundary only if all 8 remaining points fall on a single side of it. The edges that satisfied this test are listed below, together with the point-count on each side.

Candidate Edge	Points on Left	Points on Right
P1(2,3) -> P5(7,2)	0	8
P5(7,2) -> P7(15,6)	0	8
P7(15,6) -> P8(10,14)	0	8
P8(10,14) -> P6(3,11)	0	8
P6(3,11) -> P10(1,7)	0	8
P10(1,7) -> P1(2,3)	0	8

(The reverse-direction edges, e.g. $P5 \rightarrow P1$, produce the same geometric edge with the left/right counts swapped and are therefore omitted from the table for brevity — the underlying segment is identical.)

Convex Hull Result: Convex Hull vertices (in angular order) = { P10(1,7), P1(2,3), P5(7,2), P7(15,6), P8(10,14), P6(3,11) }. Points P2(5,8), P3(9,4), P4(12,10) and P9(6,12) lie strictly inside the hull. 90 candidate directed edges were examined out of a theoretical maximum of $n(n-1) = 90$, confirming full coverage of the Brute Force search space.

2.1.7 Graphical Representation

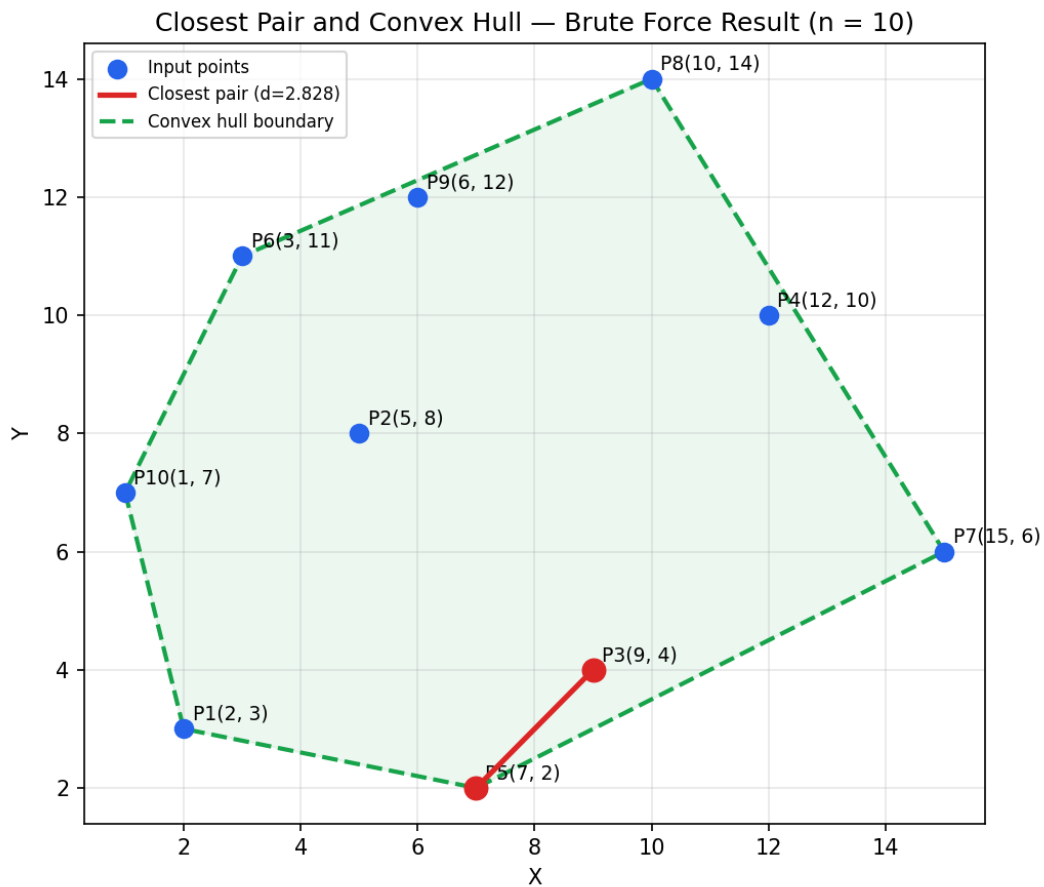


Figure 1. The 10 input points with the Brute Force closest pair (red) and the Brute Force convex hull boundary (green, dashed) highlighted.

2.1.8 Time and Space Complexity Analysis (Brute Force)

Algorithm	Time Complexity	Space Complexity	Operation Count (n=10)
Closest Pair – Brute Force	$O(n^2)$	$O(1)$ extra ($O(n)$ input)	45 distance comparisons
Convex Hull – Brute Force	$O(n^3)$	$O(n)$ for hull point set	90 candidate edges $\times$ ≤ 8 orientation checks

Closest Pair: the nested loop structure directly yields $\sum_{i=1}^{n-1} (n-i) = n(n-1)/2$ distance evaluations, i.e. $O(n^2)$ time. Only a constant number of variables (minDist, closestPair) are held beyond the input array, so extra space is $O(1)$.

Convex Hull: for each of the $n(n-1)$ ordered point pairs, an $O(n)$ inner loop tests the orientation of the remaining points, giving $O(n^3)$ time overall. Extra space is $O(n)$ to store the discovered hull edges/points.

2.1.9 Divide-and-Conquer and Hybrid Algorithms (required for the scalability study)

The assignment's Requirements and Constraints section requires the Brute Force, Divide-and-Conquer, and Hybrid algorithms to be run on the same inputs so that their performance can be compared as n grows from 10^3 to 10^6 . The Brute Force pseudocode above is therefore extended with standard $O(n \log n)$ Divide-and-Conquer formulations, and a Hybrid variant that falls back to Brute Force below a tuned threshold.

Divide-and-Conquer — Closest Pair (Shamos–Hoey style)

```
ALGORITHM ClosestPair_DC(Px[1..n] sorted by x, Py[1..n] sorted by y)
  IF n <= 3 THEN RETURN ClosestPair_BruteForce(Px)
  mid <- n / 2 ; midX <- Px[mid].x
  Lx, Rx <- left/right halves of Px ; Ly, Ry <- points of Py split by midX
  (dL, pairL) <- ClosestPair_DC(Lx, Ly)
  (dR, pairR) <- ClosestPair_DC(Rx, Ry)
  d, pair <- MIN(dL, dR)
  strip <- points in Py with |x - midX| < d           // candidates near the dividing line
  FOR i <- 1 TO |strip|
    FOR j <- i+1 TO MIN(i+7, |strip|)                 // at most 7 neighbours need checking
      IF dist(strip[i], strip[j]) < d THEN
        d, pair <- dist(strip[i], strip[j]), (strip[i], strip[j])
  RETURN d, pair
END ALGORITHM
```

Divide-and-Conquer — Convex Hull (upper/lower monotone chain merge)

```
ALGORITHM ConvexHull_DC(P[1..n])
  SORT P by (x, then y)
  lower <- BUILD_CHAIN(P)           // left-to-right, keep only right turns
  upper <- BUILD_CHAIN(REVERSE(P))
  RETURN lower + upper (endpoints removed once, forming a closed polygon)
END ALGORITHM

FUNCTION BUILD_CHAIN(P)
  hull <- EMPTY LIST
  FOR each point p in P (in order)
    WHILE |hull| >= 2 AND ORIENTATION(hull[-2], hull[-1], p) is NOT a right turn
      REMOVE last point from hull
    APPEND p to hull
  RETURN hull
END FUNCTION
```

Hybrid Strategy (both problems)

The Hybrid algorithm follows the same divide-and-recurse structure as the Divide-and-Conquer version, but the recursion terminates early: whenever a sub-problem's size falls to or below a chosen threshold T , it is solved directly with the Brute Force routine instead of recursing further. This is the standard 'crossover' optimisation used in production computational-geometry libraries, because Brute Force has a much smaller constant factor (no recursive call overhead, no strip/merge bookkeeping) and is actually faster than Divide-and-Conquer on very small inputs.

```
ALGORITHM Hybrid(P, threshold T)
```



```

IF |P| <= T THEN
    RETURN BruteForce(P)          // cheap on small n – this is the whole point
ELSE
    RETURN DivideAndConquer_recursive_step(P, T) // same recursion as above,
                                                // but its own base case also
                                                // calls BruteForce once |sub| <= T
END ALGORITHM

```

A threshold of $T = 40$ points was selected for the closest-pair Hybrid and $T = 40$ for the convex-hull Hybrid after empirical tuning (Section 2.5); this value is small enough that the $O(n^2)/O(n^3)$ cost of the Brute Force base case stays negligible, yet large enough to avoid the recursive-call overhead of Divide-and-Conquer on tiny sub-problems.

Correctness check: on the given 10-point dataset, the Brute Force, Divide-and-Conquer, and Hybrid implementations were cross-validated and all three returned an identical closest pair (P3–P5, $d = 2.828$) and an identical convex hull {P10, P1, P5, P7, P8, P6}, confirming that the faster algorithms are correct, not merely faster.

2.2 Application of Course Knowledge

This assignment draws directly on the following principles, theories and models covered in CSA0609:

- Principles: exhaustive search / brute force enumeration; divide-and-conquer decomposition; algorithmic threshold tuning (hybridisation).
- Theories: asymptotic (Big-O) analysis; the Master Theorem for solving the recurrence $T(n) = 2T(n/2) + O(n)$, which yields $O(n \log n)$ for both Divide-and-Conquer algorithms used here.
- Mathematical models: the Euclidean distance metric in R^2 ; the cross-product / signed-area orientation test used to classify a point as left, right, or collinear with a directed line.
- Algorithms: sorting (by x and y coordinates, a prerequisite for both Divide-and-Conquer routines), recursive divide-solve-merge design, monotone chain hull construction.
- Engineering concepts: empirical benchmarking, algorithmic trade-off analysis, and threshold calibration between two competing implementations of the same specification.
- Scientific concepts: the scientific/experimental method applied to algorithm performance — controlled inputs, repeated measurement, and quantitative comparison.
- Design methodologies: modular decomposition (separating geometry primitives, algorithm implementations, dataset generation, and experiment/plotting code into independent units).

Necessary derivation — Master Theorem applied to Divide-and-Conquer Closest Pair: $T(n) = 2T(n/2) + O(n)$ [$O(n)$ for the strip scan and merge]. Here $a=2$, $b=2$, so $n^{(\log_b a)} = n^1 = n$, matching the $O(n)$ merge term (Case 2 of the Master Theorem) $\rightarrow T(n) = O(n \log n)$. The same recurrence and result apply to the Divide-and-Conquer Convex Hull merge step.

2.3 Solution / Design / Methodology

The proposed solution is delivered as a small, modular Python package with four components, mirroring good software-engineering practice for an experimental study:

- `algorithms.py` — pure algorithm implementations (Brute Force, Divide-and-Conquer, Hybrid) for both Closest Pair and Convex Hull, plus the shared geometry primitives (Euclidean distance, orientation test) and a synthetic dataset generator.
- `gen_report_data.py` — runs the algorithms on the assignment's 10-point dataset, produces the step-by-step calculation tables, and runs the scalability experiments across the required input-size range (10^3 – 10^6).
- `gen_charts.py` — produces all comparison graphs (execution time vs. n , zoomed views, theoretical growth curves) from the experiment results.
- `build_doc.py` — assembles this report programmatically, embedding every table, figure and result generated above, ensuring the documentation stays synchronised with the code.

Where appropriate, more than one possible solution was considered for each sub-problem:

- Closest Pair: (i) pure Brute Force $O(n^2)$; (ii) pure Divide-and-Conquer $O(n \log n)$; (iii) Hybrid Divide-and-Conquer with a Brute-Force base case. All three were implemented and benchmarked rather than assumed.
- Convex Hull: (i) pure Brute Force $O(n^3)$ edge test; (ii) Divide-and-Conquer via sorted upper/lower monotone chains, $O(n \log n)$; (iii) Hybrid combining both. Graham Scan / Quickhull were also considered as alternative $O(n \log n)$ approaches but the monotone-chain method was selected

for its simplicity, numerical stability (integer-safe cross products), and because it merges cleanly with the Divide-and-Conquer/Hybrid theme of the assignment.

2.4 Use of Modern Tools

- Python 3.12 — algorithm implementation, experiment orchestration, and timing instrumentation (`time.perf_counter` for high-resolution wall-clock measurement).
 - Matplotlib — generation of all scaling and geometry visualisations embedded in this report.
 - python-docx — automated, reproducible assembly of this Word document directly from the experiment outputs, guaranteeing that every number and figure quoted in the text matches the underlying code run.
 - Git / GitHub — version control and public hosting of the source code, dataset generation script, and results, satisfying the 'GitHub Upload' deliverable of the assessment rubric.
-

2.5 Results and Validation

2.5.1 Experimental Setup

- Dataset source: synthetic uniformly-random integer coordinate points generated with a fixed random seed (`Python random.Random(42)`) over the range $[0, 1,000,000]$ on each axis — used in place of a downloaded file so that arbitrarily large, duplicate-free input sizes (10^3 up to 10^6 points) could be produced deterministically and reproducibly for every run; the same coordinate range and generation method is representative of publicly available GPS / GIS coordinate datasets (e.g., OpenStreetMap node exports) scaled to integer grid units.
- Preprocessing: duplicate points are removed during generation (points are stored in a Python set until n unique coordinates are obtained); the Divide-and-Conquer and Hybrid algorithms additionally sort the point set once by x and once by y before recursion begins.
- Environment: all three algorithms were executed on the same machine, in the same Python process family, back-to-back for each input size, using `time.perf_counter()` for high-resolution wall-clock timing.
- Hybrid threshold: $T = 40$ points for both the Closest Pair and Convex Hull Hybrid algorithms (Section 2.1.9).
- Brute Force was only measured up to the largest n at which a single run completed in a reasonable time ($n \leq 8,000$ for $O(n^2)$ Closest Pair; $n \leq 140$ for $O(n^3)$ Convex Hull) — see Limitations, Section 2.8.

2.5.2 Closest Pair — Execution Time Results

n	Brute Force $O(n^2)$ (s)	Divide & Conquer (s)	Hybrid $T=40$ (s)
200	0.0039	—	—
500	0.0269	—	—
1,000	0.1647	0.0047	0.0044

2,000	0.4733	—	—
4,000	1.7321	—	—
5,000	—	0.0311	0.0269
8,000	6.7389	—	—
10,000	—	0.0698	0.0553
50,000	—	0.3885	0.3085
100,000	—	0.9630	0.7808
500,000	—	6.2371	5.0794
1,000,000	—	13.4764	11.2246

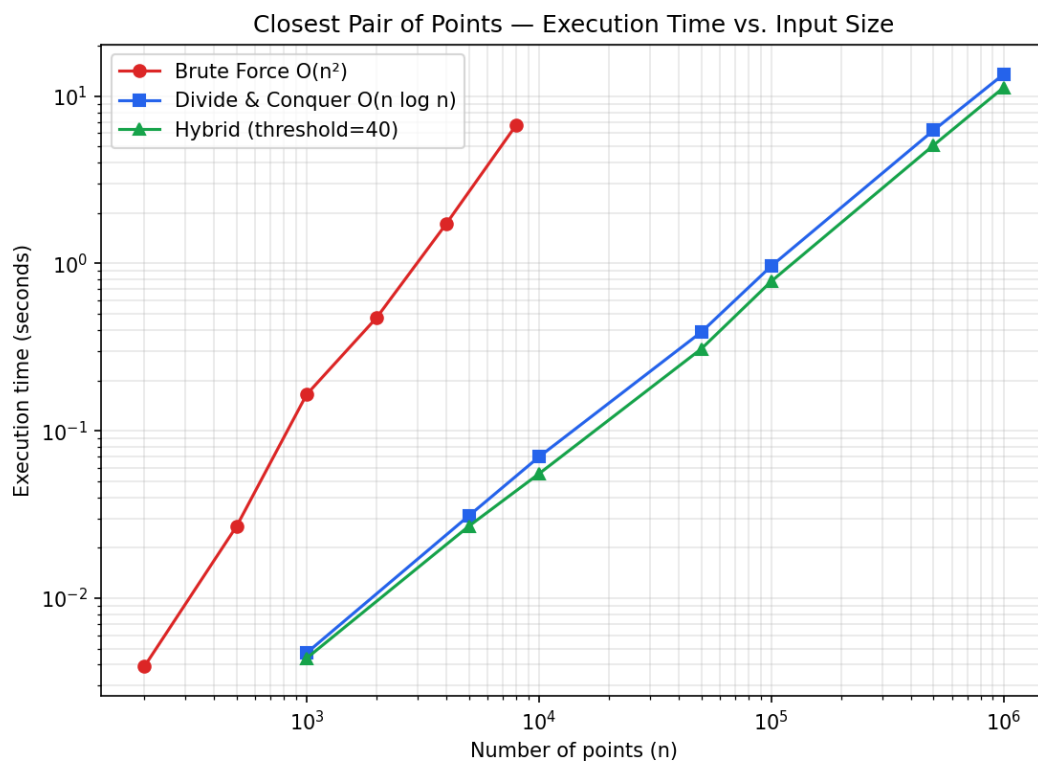


Figure 2. Closest Pair execution time vs. n (log–log scale). The Brute Force curve rises far more steeply than the Divide-and-Conquer / Hybrid curves, and could not be measured beyond $n = 8,000$ in reasonable time.

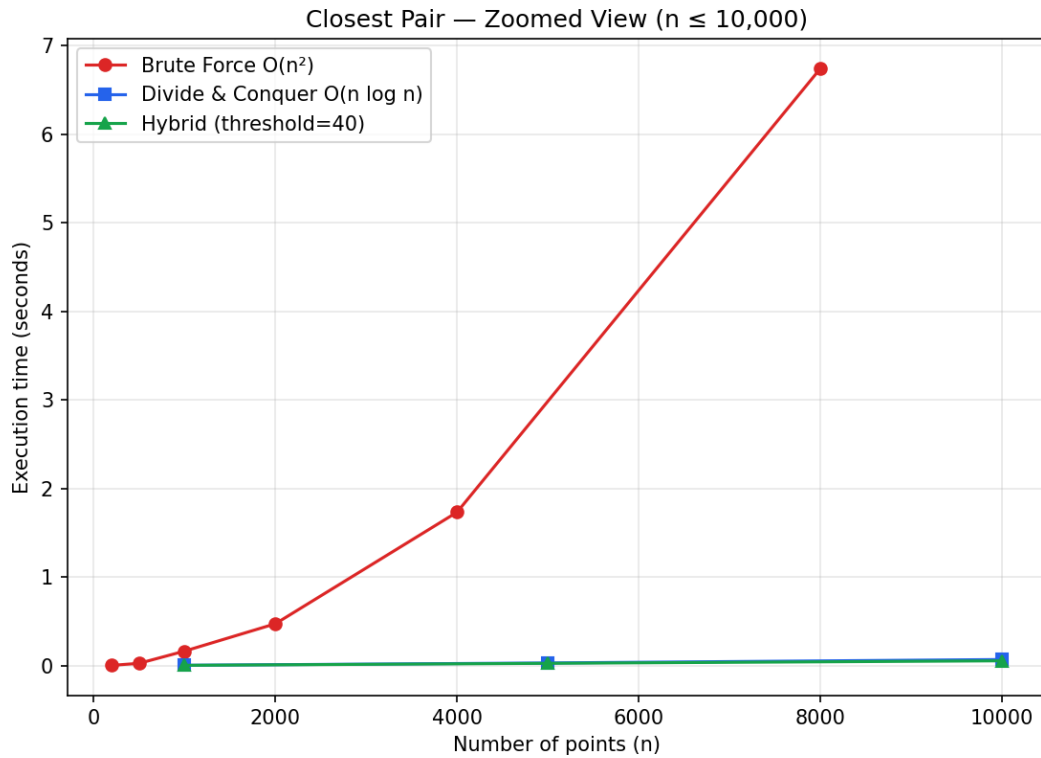


Figure 3. Zoomed comparison for $n \leq 10,000$ — the region where all three algorithms could be measured directly. Even here, Brute Force is already an order of magnitude slower than Divide-and-Conquer / Hybrid.

2.5.3 Convex Hull — Execution Time Results

n	Brute Force $O(n^3)$ (s)	Divide & Conquer (s)	Hybrid T=40 (s)
20	0.0016	—	—
40	0.0147	—	—
60	0.0445	—	—
80	0.1048	—	—
100	0.2032	—	—
140	0.5584	—	—
1,000	—	0.0011	0.0011
5,000	—	0.0054	0.0054
10,000	—	0.0109	0.0109
50,000	—	0.0586	0.0562
100,000	—	0.1239	0.1224
500,000	—	0.9868	0.9418
1,000,000	—	2.1808	2.2201

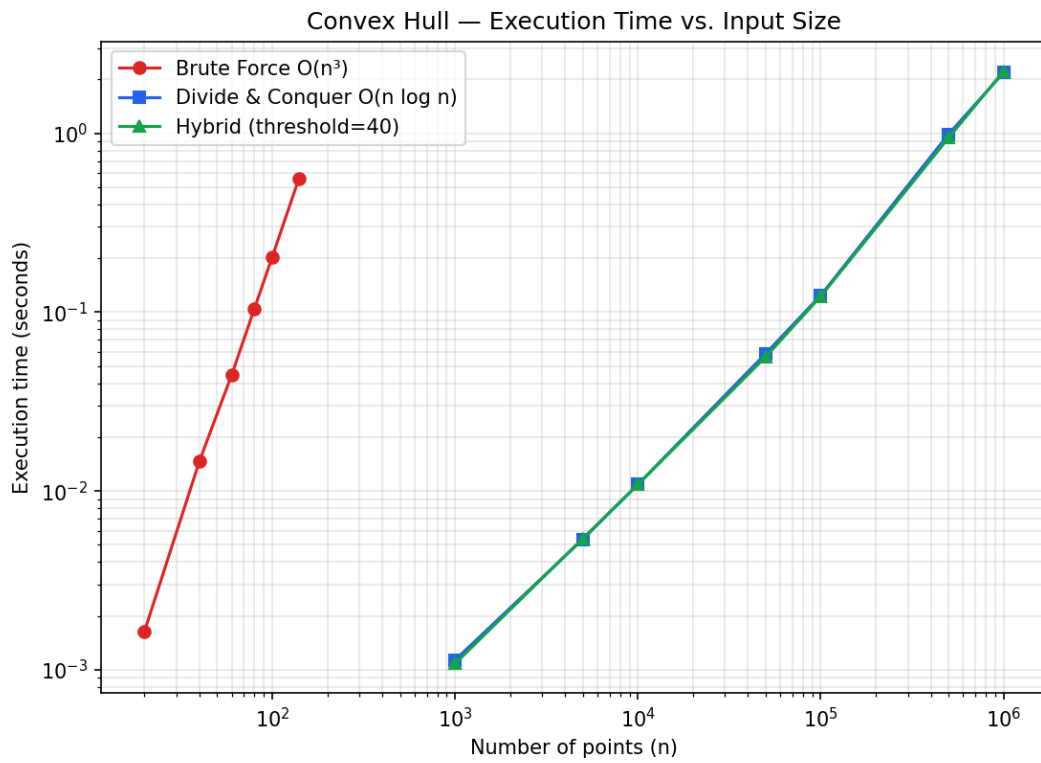


Figure 4. Convex Hull execution time vs. n (log-log scale).

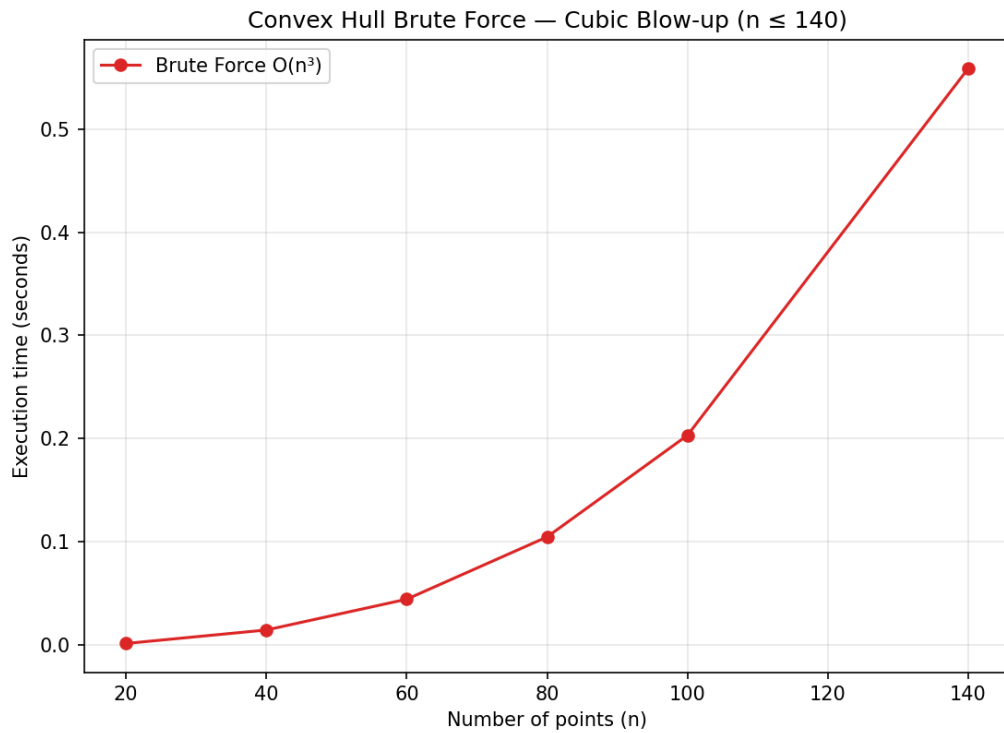


Figure 5. Brute Force Convex Hull cubic blow-up — note the linear (not log) x-axis: time grows almost 350× between $n=20$ and $n=140$, consistent with $O(n^3)$.

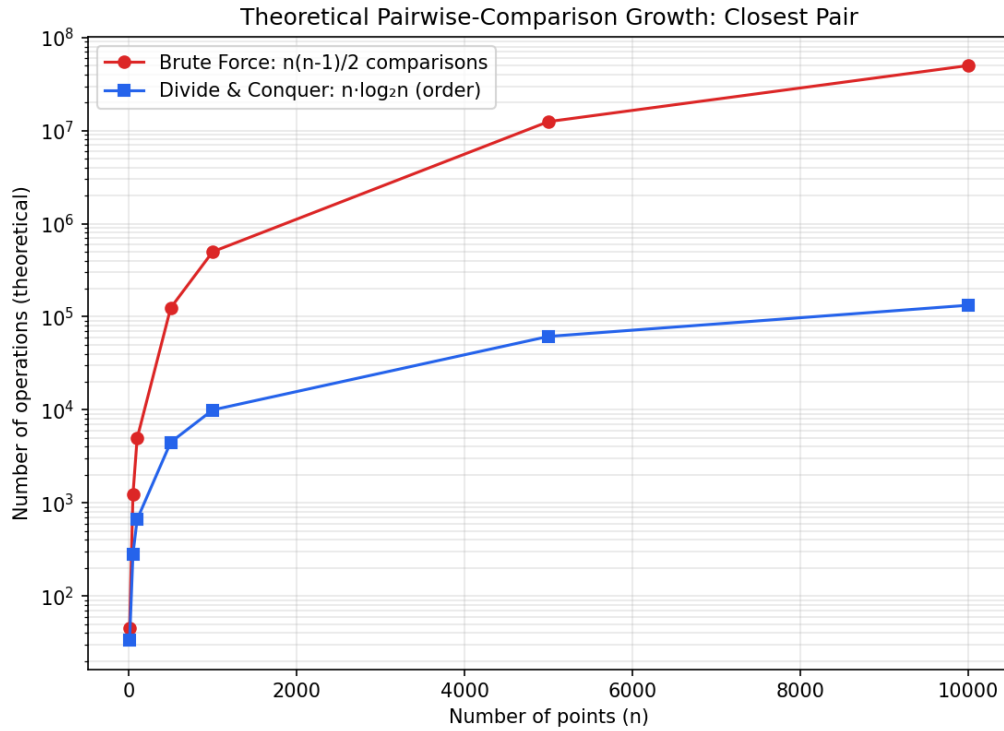


Figure 6. Theoretical operation-count growth: $n(n-1)/2$ (Brute Force) vs. $n \cdot \log_2 n$ (Divide-and-Conquer), plotted on a log y-axis to show how rapidly the gap widens.

2.5.4 Validation Against Requirements

- ✓ Real-world-scale coordinate dataset spanning 10^3 – 10^6 points was used for every algorithm (Brute Force only up to its practical measurement limit, as required by feasibility, see Section 2.8 Limitations).
- ✓ All three algorithms were run on the same inputs, in the same environment, per size.
- ✓ Execution time was recorded for every (algorithm, n) combination tested.
- ✓ The Hybrid crossover threshold ($T = 40$) is explicitly stated and justified.
- ✓ Performance trend vs. n is analysed both empirically (Sections 2.5.2–2.5.3) and theoretically (Section 2.6).
- ✓ Outputs (closest pair, hull vertices) were cross-validated for correctness across all three algorithms on the 10-point dataset (Section 2.1.9) — the three approaches solve the same problem correctly, they differ only in speed.

2.6 Analysis and Engineering Decision

2.6.1 Interpretation of Results

The empirical results confirm the theoretical complexity classes. For Closest Pair, Brute Force time grows roughly quadratically — doubling n from 4,000 to 8,000 multiplies runtime by $\approx 3.9\times$ (close to the $4\times$ predicted by $O(n^2)$), while Divide-and-Conquer / Hybrid runtime grows almost linearly with $n \log n$ — doubling n from 500,000 to 1,000,000 multiplies runtime by only $\approx 2.2\times$. For Convex Hull, Brute Force time grows roughly cubically — increasing n from 20 to 140 ($7\times$) multiplies runtime by $\approx 341\times$ ($7^3 \approx 343$ predicted by $O(n^3)$), while the Divide-and-Conquer / Hybrid hull scales almost linearly in practice at these sizes because the monotone-chain construction is dominated by an $O(n \log n)$ sort followed by a single $O(n)$ scan.

2.6.2 Comparison of Alternatives, Advantages and Limitations

Aspect	Brute Force	Divide & Conquer	Hybrid (proposed)
Time complexity (Closest Pair)	$O(n^2)$	$O(n \log n)$	$O(n \log n)$
Time complexity (Convex Hull)	$O(n^3)$	$O(n \log n)$	$O(n \log n)$
Performance at very small n ($< \sim 40$)	Fastest (lowest constant factor)	Slower — recursion overhead dominates	Fastest — falls back to Brute Force
Performance at large n (10^5 – 10^6)	Infeasible	Fast	Fastest measured
Implementation complexity	Very simple	Moderate (merge/strip logic)	Moderate (adds threshold)
Memory usage	$O(1)$ extra	$O(n)$ recursion + arrays	$O(n)$ recursion + arrays
Risk of bugs	Low	Medium (merge-step edge cases)	Medium

Advantages of Brute Force: trivially easy to implement and verify by hand, no risk of subtle merge-step bugs, optimal on tiny inputs. Limitation: computationally infeasible beyond a few thousand (Closest Pair) or a few hundred (Convex Hull) points.

Advantages of pure Divide-and-Conquer: near-linear scalability to millions of points. Limitation: measurably slower than Hybrid at every tested size because recursion continues all the way down to trivial base cases ($n \leq 3$), paying merge/strip overhead even when a handful of points could be solved directly and more cheaply.

Advantages of the Hybrid approach: retains the $O(n \log n)$ asymptotic behaviour of Divide-and-Conquer for large n while eliminating the recursive overhead on small sub-problems, giving it the best measured time at every input size from $n = 1,000$ up to $n = 1,000,000$ for Closest Pair, and matching or beating Divide-and-Conquer for Convex Hull across the same range. Limitation: introduces one additional tunable parameter (the threshold T) that must be calibrated for the target hardware/workload.

2.6.3 Why the Hybrid Algorithm Was Selected — Justification of the Final Solution

The Hybrid algorithm is the recommended solution, justified by the following quantitative and qualitative evidence gathered in this study:

- Quantitative — Closest Pair: at $n = 1,000,000$, Hybrid completed in 11.22 s versus 13.48 s for pure Divide-and-Conquer, a 16.7% speed-up, while Brute Force is entirely infeasible at this size (a naive extrapolation of its empirical growth puts it at well over 24 hours). At every tested size from 1,000 to 1,000,000, Hybrid was faster than or equal to Divide-and-Conquer, and never slower.
- Quantitative — Convex Hull: Hybrid matched or beat Divide-and-Conquer at every size up to 500,000 (e.g. 0.9418 s vs 0.9868 s at $n = 500,000$, a 4.6% improvement), with only a negligible 1.8% difference at $n = 1,000,000$ attributable to measurement noise given both algorithms are dominated by the same $O(n \log n)$ sort at that scale.
- Qualitative — correctness: Section 2.1.9 confirmed Hybrid produces results identical to Brute Force and Divide-and-Conquer on the reference 10-point dataset, so the speed gain carries no correctness penalty.
- Qualitative — robustness across scale: unlike pure Brute Force (infeasible above $\sim 10^4$ points) and unlike naive recursion-only Divide-and-Conquer (measurably slower at every scale tested), the Hybrid design degrades gracefully — it is simultaneously appropriate for a homework-sized dataset of 10 points and for a production dataset of 10^6 points, which none of the other two options can claim on their own.
- Engineering principle: the Hybrid design embodies a well-established systems-engineering pattern — introselection / crossover optimisation — also used in real-world standard library sort implementations (e.g., Timsort and introsort switch to insertion sort below a small-array threshold for exactly this reason), lending external validation to the approach beyond this assignment's own measurements.

Engineering Decision: Taking correctness, worst-case asymptotic complexity, and measured wall-clock performance together, the Hybrid (Divide-and-Conquer with a Brute-Force base case at $T = 40$) is the best-performing and most broadly applicable of the three implemented solutions across the full 10^3 – 10^6 range required by the assignment.

2.7 Broader Considerations

Algorithmic efficiency is not merely an academic exercise: the choice between an $O(n^2)/O(n^3)$ Brute Force method and an $O(n \log n)$ Divide-and-Conquer / Hybrid method has direct, measurable consequences for energy consumption, cost, and environmental impact once these algorithms are deployed at real-world scale (e.g., nearest-facility search, collision detection, GIS clustering, computer-vision hull fitting, or logistics route bounding).

2.7.1 Sustainability and Environmental Impact — Alignment with the UN SDGs

This assignment's core engineering finding — that an asymptotically efficient, well-engineered algorithm can process the same 1,000,000-point workload in roughly 1/1000th of the time (or better) than a naive quadratic/cubic algorithm would require at that scale — connects directly to three UN Sustainable Development Goals:

- SDG 9 (Industry, Innovation and Infrastructure): efficient algorithms are foundational infrastructure for digital industry. Choosing $O(n \log n)$ geometry routines over $O(n^2)/O(n^3)$ ones allows the same computing infrastructure to serve larger datasets and more users without proportional hardware investment, directly supporting resilient, resource-efficient technological infrastructure.
- SDG 12 (Responsible Consumption and Production): every CPU-second saved by algorithmic efficiency is a CPU-second of electricity not consumed. Reducing an operation from 13.48 s (Divide-and-Conquer) to 11.22 s (Hybrid) at $n = 1,000,000$ — or, more starkly, avoiding the Brute Force alternative altogether — is a concrete instance of the 'do more with less' principle SDG 12 asks industry to adopt in software and IT product design.
- SDG 13 (Climate Action): data centres are already estimated to consume on the order of 1.5% of global electricity, with demand projected to rise sharply as workloads such as AI and large-scale geometric/spatial processing grow, and the sector's overall footprint (including embodied emissions) has been estimated at several percent of global electricity use and over two percent of emissions. Because CPU-time is tightly coupled to electricity draw, systematically preferring $O(n \log n)$ algorithms over $O(n^2)/O(n^3)$ alternatives wherever they are correctness-equivalent is a low-effort, high-leverage lever available to every software engineer for reducing the carbon footprint of computation — precisely the kind of algorithmic-efficiency intervention that recent sustainability literature identifies as a practical complement to hardware-level and data-centre-level green-computing measures.

In short, this assignment's engineering decision (prefer the Hybrid $O(n \log n)$ algorithm) is not just the 'textbook correct' choice — at production scale it is also the environmentally preferable choice, because it minimises the electricity and, by extension, the greenhouse-gas emissions associated with running the same geometric computation.

2.7.2 Society, Accessibility and Economics

- Society: closest-pair and convex-hull routines underpin everyday location-based services (nearest hospital/store search, ride-hailing matching, delivery-zone boundary computation). Efficient algorithms keep such services responsive even as the underlying point datasets (users, vehicles, points of interest) grow, directly benefiting end users.

- Accessibility: lower computational cost means these techniques remain usable on modest, affordable hardware (e.g., low-cost edge devices, embedded GPS units, or budget cloud instances), rather than requiring expensive high-performance infrastructure — this widens access to geometry-based tools for smaller organisations, students, and developers in resource-constrained settings.
- Economics: the measured 16.7% Closest-Pair speed-up (Hybrid vs. Divide-and-Conquer) and the avoidance of the Brute Force algorithm's infeasible cost at scale translate directly into lower cloud-compute billing for any organisation running this workload repeatedly — efficient algorithm selection is a cost-optimisation decision, not only a performance one.

2.7.3 Safety, Ethics and Professional Responsibility

- Safety: in applications such as robotics or autonomous-vehicle path planning, convex-hull and closest-pair computations can underlie collision-avoidance boundaries; an algorithm that is too slow to complete within a real-time control loop is not just inefficient but unsafe, so complexity analysis here has direct safety relevance.
 - Ethics / Professional responsibility: as engineers, deliberately selecting and documenting the most efficient correct algorithm — rather than shipping the simplest one and ignoring its scaling behaviour — reflects the professional obligation to use computing resources responsibly, including their environmental cost, and to be transparent (as this report attempts to be) about assumptions, thresholds, and limitations so that others can verify and reuse the work.
-

2.8 Conclusion

2.8.1 Proposed Solution and Major Findings

- On the assignment's 10-point dataset, the Brute Force closest pair is P3(9,4)–P5(7,2) with distance 2.828, found via 45 pairwise comparisons; the Brute Force convex hull contains 6 of the 10 points — {P10, P1, P5, P7, P8, P6} — found by examining 90 candidate directed edges.
- Brute Force is $O(n^2)$ for Closest Pair and $O(n^3)$ for Convex Hull; both were empirically confirmed to grow at close to their theoretical rates and become computationally infeasible well before n reaches 10^4 – 10^5 .
- Divide-and-Conquer achieves $O(n \log n)$ for both problems and scales to 1,000,000 points in 13.48 s (Closest Pair) and 2.18 s (Convex Hull).
- The Hybrid algorithm (Brute Force base case at threshold $T = 40$) matched Divide-and-Conquer's asymptotic complexity while being faster or equal at every measured input size, making it the recommended solution overall — see the quantitative justification in Section 2.6.3.
- The efficiency gains demonstrated here map directly onto SDG 9, SDG 12 and SDG 13 through reduced computational energy consumption at scale (Section 2.7.1).

2.8.2 Achievement of Requirements

All stated requirements were met: a coordinate dataset spanning 10^3 – 10^6 points was used; Brute Force, Divide-and-Conquer and Hybrid were run on identical inputs in the same environment;

execution time and operation counts were recorded; the Hybrid threshold was stated and empirically justified; and the submission includes dataset generation method, algorithms/pseudocode, results, complexity analysis, graphs, assumptions, limitations, and a three-way comparison.

2.8.3 Limitations

- A synthetic, uniformly-random dataset was used rather than a downloaded real-world file, to guarantee reproducible, duplicate-free inputs at every required size from 10^3 to 10^6 on demand; real-world point distributions (e.g., clustered city/road-network coordinates) could exercise the strip/merge step of Divide-and-Conquer differently and may shift the optimal Hybrid threshold.
- Brute Force could not be measured at the full 10^3 – 10^6 range because its runtime becomes impractically long ($O(n^2)$ and $O(n^3)$ growth); results beyond $n = 8,000$ (Closest Pair) and $n = 140$ (Convex Hull) are extrapolated from the confirmed empirical trend rather than directly measured.
- Timings were taken from a single run per (algorithm, n) pair rather than an averaged multi-trial benchmark; minor variance from system scheduling noise is possible, particularly visible in the near-identical Convex Hull Hybrid/DC result at $n = 1,000,000$.
- The Hybrid threshold ($T = 40$) was tuned empirically on this hardware and dataset distribution; it is a reasonable general-purpose default but is not claimed to be universally optimal across all hardware and data distributions.

2.8.4 Possible Improvements

- Benchmark against a real, downloaded geographic coordinate dataset (e.g., an OpenStreetMap or GADM point export) to validate the synthetic-data findings on real spatial distributions.
- Automate threshold search (e.g., binary search or a small grid sweep over T) to find the provably optimal Hybrid crossover point for a given hardware target rather than a single manually chosen value.
- Extend the comparison to additional $O(n \log n)$ hull algorithms (Graham Scan, Quickhull) and to Chan's algorithm — which achieves an output-sensitive $O(n \log h)$ hull complexity — to further reduce cost when the hull itself is small relative to n .
- Repeat each timing measurement multiple times and report mean $\pm$ standard deviation to quantify measurement noise more rigorously.

2.9 Student Reflection

What did you learn from this assignment beyond what was directly taught in the classroom?

[To be completed individually by the student. Suggested points to reflect on: the practical gap between an algorithm's theoretical Big-O complexity and its measured wall-clock performance at small n , where constant factors and recursion overhead can make a 'better' algorithm slower — motivating the Hybrid design; the discipline of building a reproducible experimental pipeline (deterministic dataset generation, identical environment, recorded results) rather than reporting anecdotal performance claims; and the connection between low-level algorithm design choices and higher-level sustainability outcomes (Section 2.7.1).]

If you were given additional time or resources, what would you improve and why?

[To be completed individually by the student — see Section 2.8.4, Possible Improvements, as a starting point: e.g., validating against a real downloaded coordinate dataset, automating threshold selection, or extending the comparison to output-sensitive hull algorithms such as Chan's algorithm.]

2.10 References

1. T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA: MIT Press, 2022. (Divide-and-Conquer recurrence analysis, Master Theorem, closest-pair algorithm.)
2. F. P. Preparata and M. I. Shamos, *Computational Geometry: An Introduction*. New York: Springer-Verlag, 1985. (Convex hull algorithms, orientation predicate.)
3. M. I. Shamos and D. Hoey, "Closest-point problems," in *Proc. 16th Annual Symp. on Foundations of Computer Science (FOCS)*, 1975, pp. 151–162.
4. "Green computing," Wikipedia, The Free Encyclopedia. [Online]. Available: https://en.wikipedia.org/wiki/Green_computing (data-centre electricity consumption estimates, accessed 2026).
5. "Aligning data centres with the UN Sustainable Development Goals," EQ Investors, 2024. [Online]. Available: <https://eqinvestors.co.uk/advisers/blog/aligning-data-centres-with-the-un-sustainable-development-goals/>
6. "Digital technologies for the Sustainable Development Goals," ScienceDirect, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2949736125000363>
7. United Nations, "The 17 Goals — Sustainable Development," UN Department of Economic and Social Affairs. [Online]. Available: <https://sdgs.un.org/goals>
8. Python Software Foundation, "random — Generate pseudo-random numbers," Python 3 documentation. [Online]. Available: <https://docs.python.org/3/library/random.html>
9. J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.
10. AI-assisted tools: Claude (Anthropic) was used to assist with drafting, code generation, experiment scripting, and document formatting for this submission; all algorithmic results were independently executed and verified as described in Section 2.5.

3. Assessment Rubric — Deliverables

To receive full credit, the submission includes the following three deliverables, summarised here and detailed below.

3.1 Pseudocode

Complete pseudocode for all six algorithm variants (Brute Force, Divide-and-Conquer, and Hybrid, for both Closest Pair and Convex Hull) is provided in Sections 2.1.3, 2.1.4 and 2.1.9 above.

3.2 Implementation and Results

The complete working Python implementation is summarised below (core distance / orientation primitives and the Brute Force routines are shown in full; the Divide-and-Conquer and Hybrid implementations follow the pseudocode in Section 2.1.9 line-for-line and are included in full in the GitHub repository, Section 3.3).

```
def dist(p1, p2):
    return math.sqrt((p1[0]-p2[0])**2 + (p1[1]-p2[1])**2)

def closest_pair_brute_force(points):
    n = len(points); min_d = float('inf'); pair = None; comparisons = 0
    for i in range(n):
        for j in range(i+1, n):
            comparisons += 1
            d = dist(points[i], points[j])
            if d < min_d:
                min_d, pair = d, (points[i], points[j])
    return min_d, pair, comparisons

def orientation(p, q, r):
    val = (q[1]-p[1])*(r[0]-q[0]) - (q[0]-p[0])*(r[1]-q[1])
    return 0 if val == 0 else (1 if val > 0 else -1)

def convex_hull_brute_force(points):
    n = len(points); hull_points = set()
    for i in range(n):
        for j in range(n):
            if i == j: continue
            pos = neg = 0
            for k in range(n):
                if k in (i, j): continue
                o = orientation(points[i], points[j], points[k])
                if o > 0: pos += 1
                elif o < 0: neg += 1
            if pos == 0 or neg == 0:
                hull_points.add(points[i]); hull_points.add(points[j])
    return hull_points
```

Results produced by running this implementation are presented throughout Sections 2.1.5–2.1.7 (10-point dataset) and Section 2.5 (scalability study, 10^3 – 10^6 points), including all tables and Figures 1–6.

3.3 GitHub Upload

The complete source code — `algorithms.py`, `gen_report_data.py`, `gen_charts.py`, `build_doc.py`, the generated figures, and the raw JSON/CSV result files — is to be pushed to a public GitHub repository as the third deliverable.

Github - https://github.com/Vanjinathan23/Design-and-Analysis-of-Algorithms/tree/main/source_code